JAVA - SPRINGBOOT

¿Qué es un servidor web?

Un servidor web es un software o hardware que recibe y responde solicitudes HTTP de los clientes (navegadores o aplicaciones). Su función principal es entregar contenido web, como páginas HTML, imágenes, videos y datos en formato JSON o XML.

Funcionamiento básico:

1. Un cliente (navegador) envía una solicitud HTTP a una URL.
2. El servidor web recibe la solicitud y procesa la información.
3. Devuelve una respuesta (una página web, un archivo o datos en JSON).
4. El cliente interpreta y muestra la información.

¿Qué es Spring Boot?

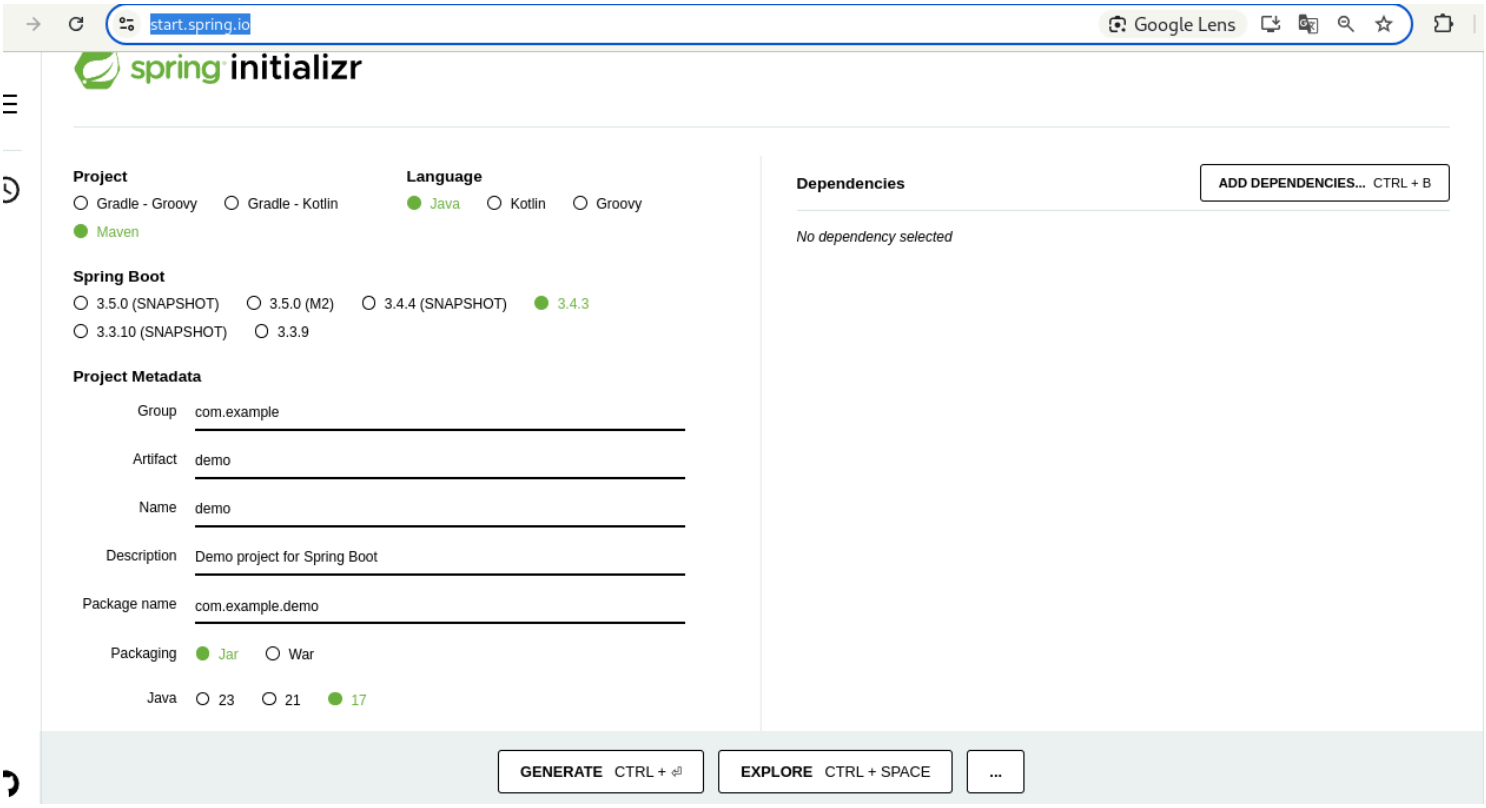
Spring Boot es un **framework de Java** basado en Spring que permite crear aplicaciones web y APIs de manera rápida y con una configuración mínima.

Características de Spring Boot:

Facilita la configuración: Usa convenciones en lugar de configuraciones manuales.
Manejo de dependencias simplificado: Usa **Spring Boot Starters** para incluir bibliotecas necesarias.
Compatible con bases de datos: Mysql, Postgres, Oracle entre otras muchas.

Creando una app SpringBoot

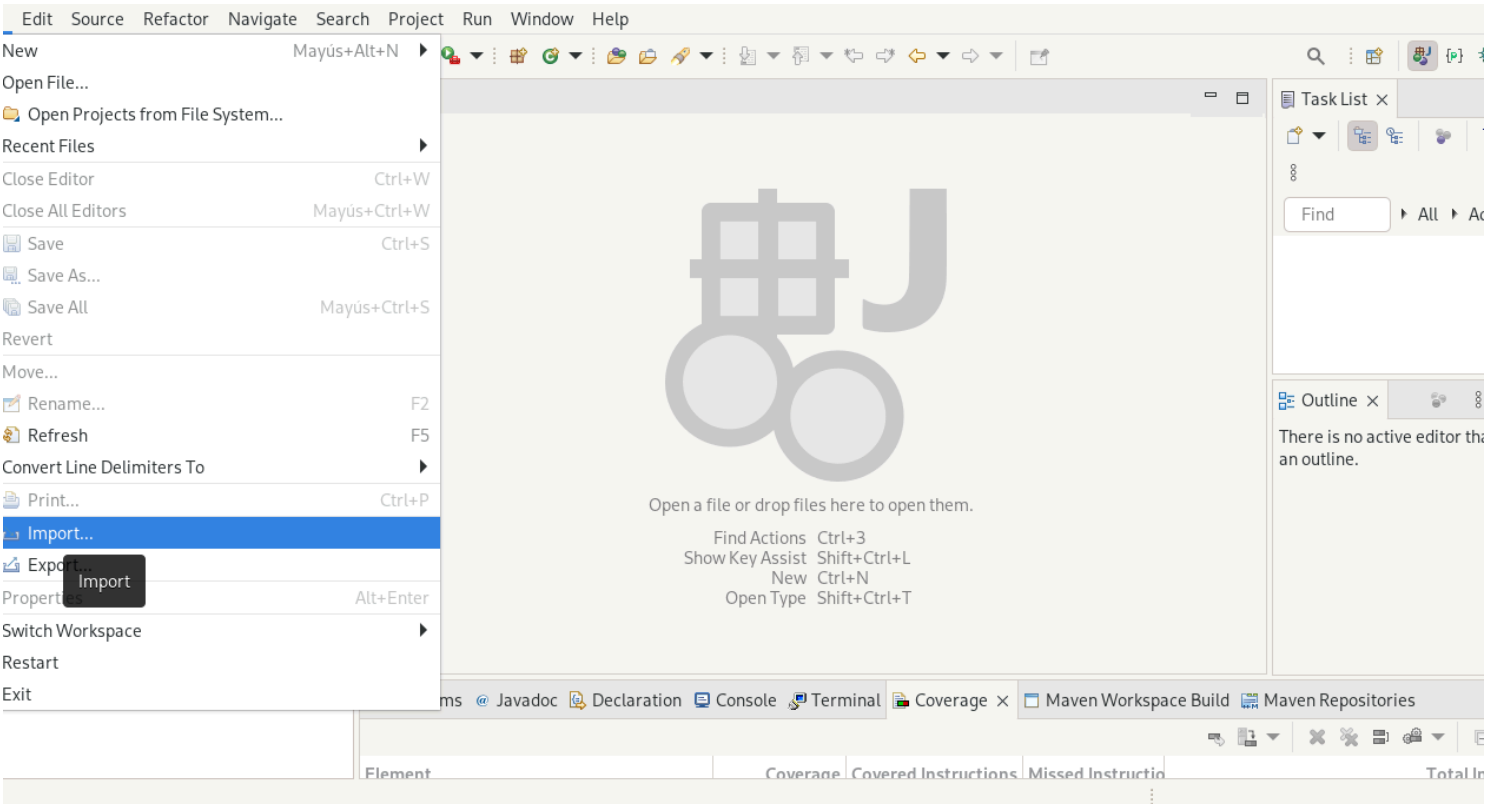
Para facilitar la creación de un proyecto SpringBoot y que tengamos la estructura inicial y dependencias, usamos https://start.spring.io/



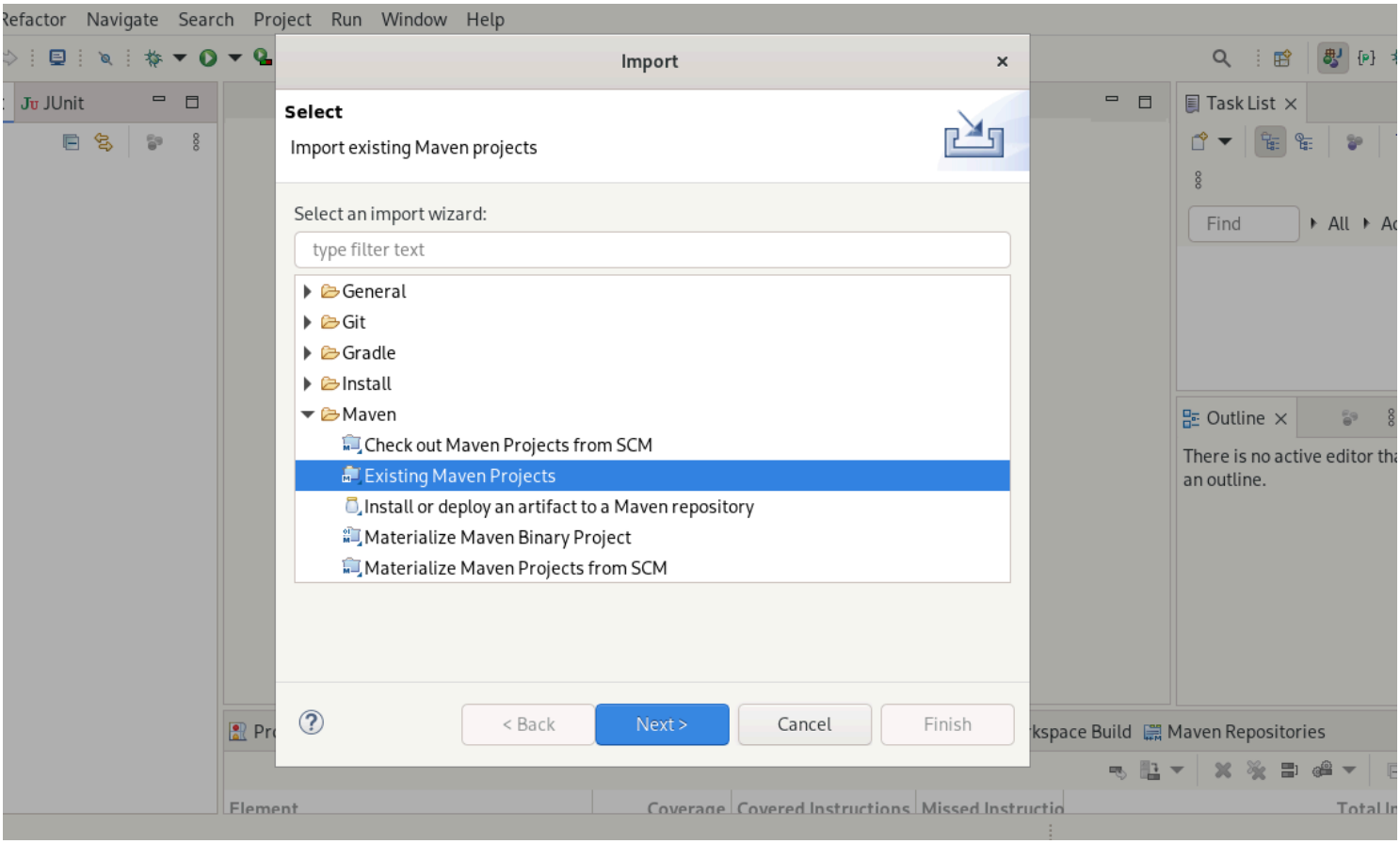
click en GENERATE

ESto descargará una carpeta con un proyecto SpringBoot

Vamos a Eclipse y hacemos click en import

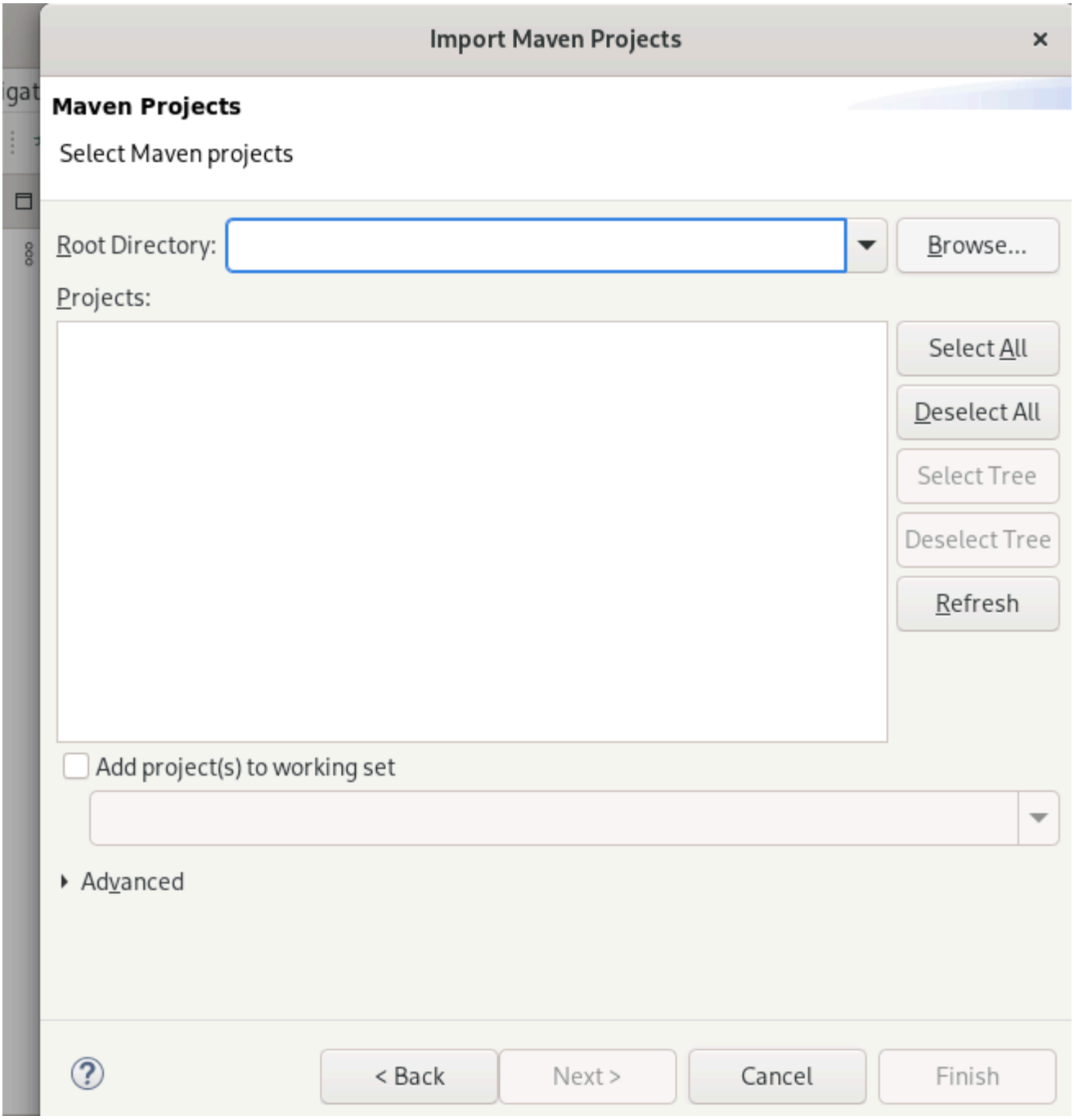


se abre la siguiente ventana

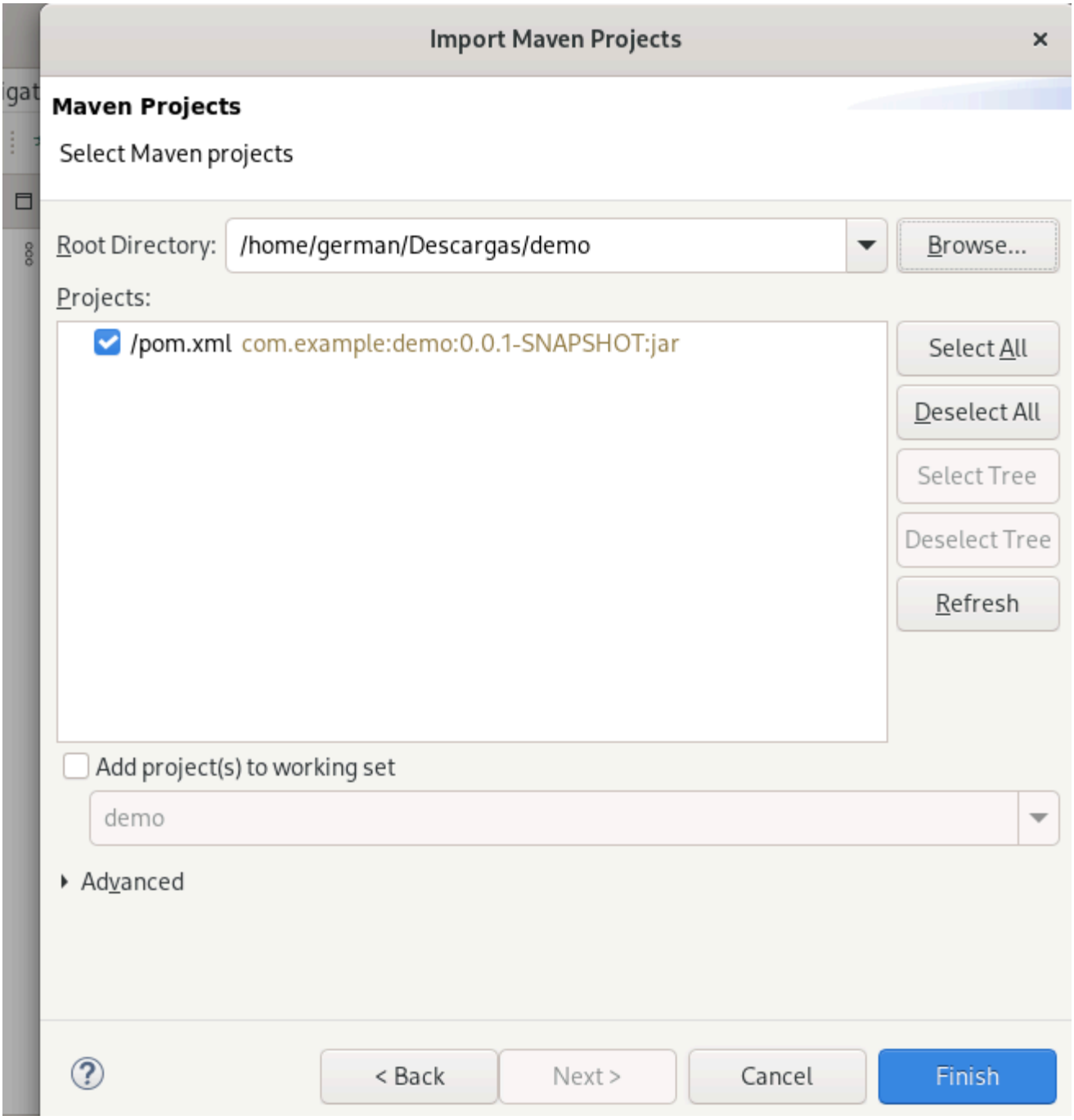


hacemos click en Next

se abre la ventana

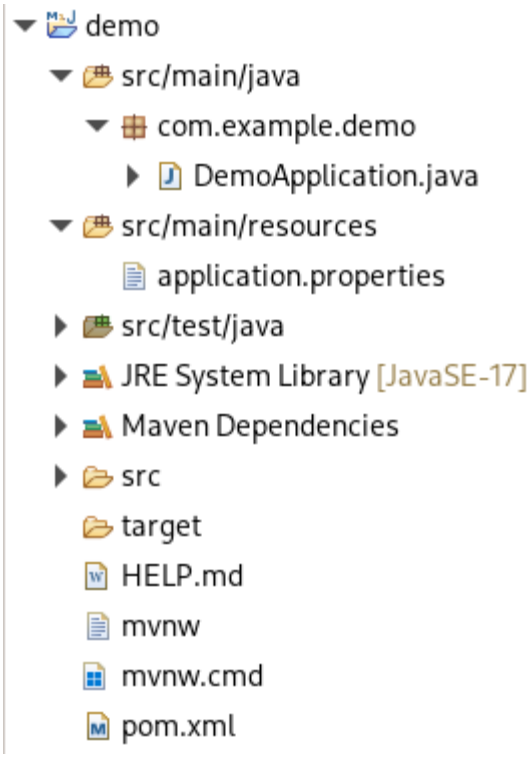


y hacemos click en Browser y buscamos la carpeta que descargamos desde https://start.spring.io/



hacemos click en finish

Abrimos el nuevo proyecto y tenemos:



Acá observamos

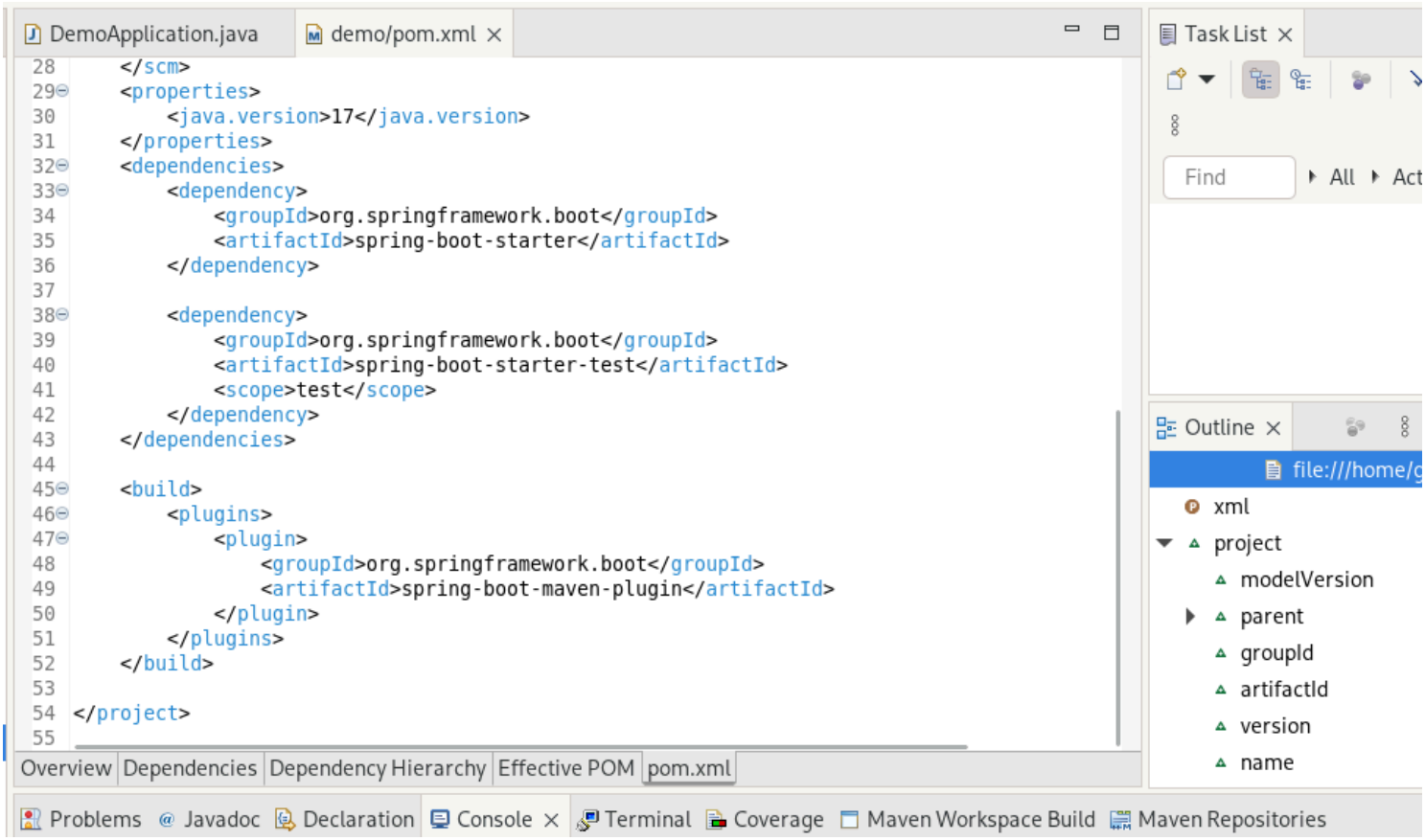
src/main - Este directorio contiene el código fuente principal de la aplicación:

- src/main/java - Código fuente Java de la aplicación
- src/main/resources - Recursos no-Java como archivos de configuración, propiedades, XML, etc.
- src/main/webapp - Para aplicaciones web, contiene archivos web como HTML, CSS, JS, JSP, etc.

src/test - Este directorio contiene el código fuente para pruebas:

- src/test/java - Código fuente Java para pruebas unitarias
- src/test/resources - Recursos necesarios para las pruebas

También observamos un archivo pom.xml



esto quiere decir que nuestro proyecto trabaja con un gestor de dependencias llamado Maven, el cual , facilita la descarga de dependencias de nuestro proyecto. Observamos que en las dependencias de nuestro pom aparecen:

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
</dependency>
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
</dependency>
```

estas dependencias en el pom, al construir el proyecto, se descargan e instalan automáticamente, lo que facilita mucho el trabajo.

Actualicemos nuestro archivo pom a:

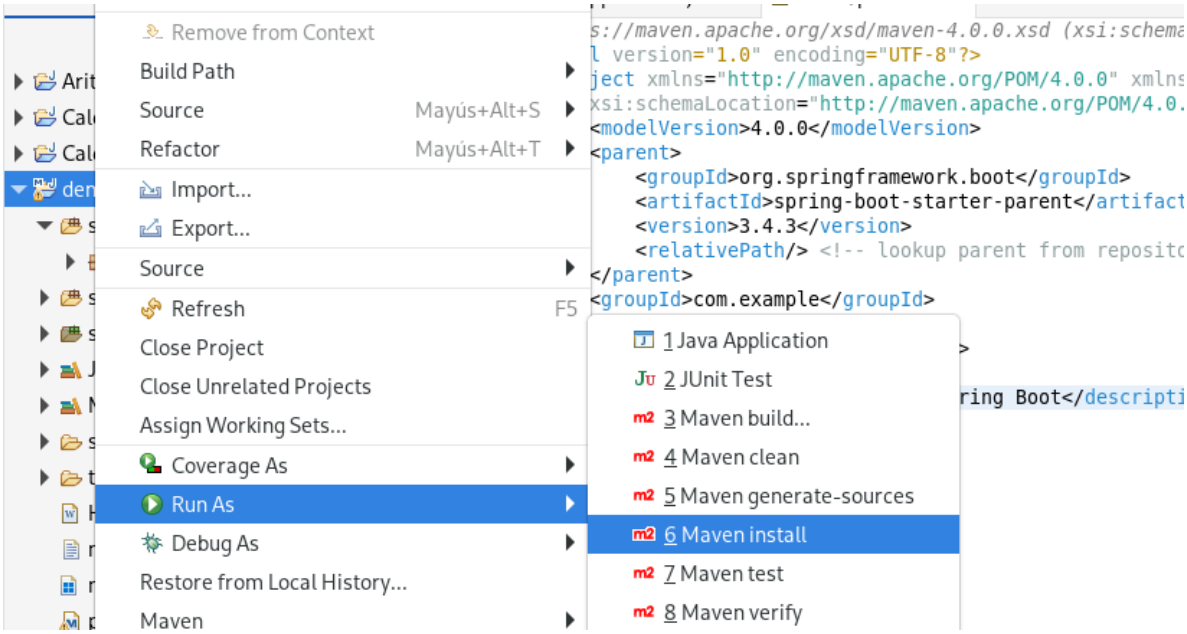
```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<parent>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-parent</artifactId>
		<version>3.4.3</version>
		<relativePath/> <!-- lookup parent from repository -->
	</parent>
	<groupId>com.example</groupId>
	<artifactId>demo</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<name>demo</name>
	<description>Demo project for Spring Boot</description>
	<url/>
	<licenses>
		<license/>
	</licenses>
	<developers>
		<developer/>
	</developers>
	<scm>
		<connection/>
		<developerConnection/>
		<tag/>
		<url/>
	</scm>
	<properties>
		<java.version>17</java.version>
	</properties>
	<dependencies>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter</artifactId>
		</dependency>

		<dependency>
	 <groupId>org.springframework.boot</groupId>
	 <artifactId>spring-boot-starter-web</artifactId>
	</dependency>

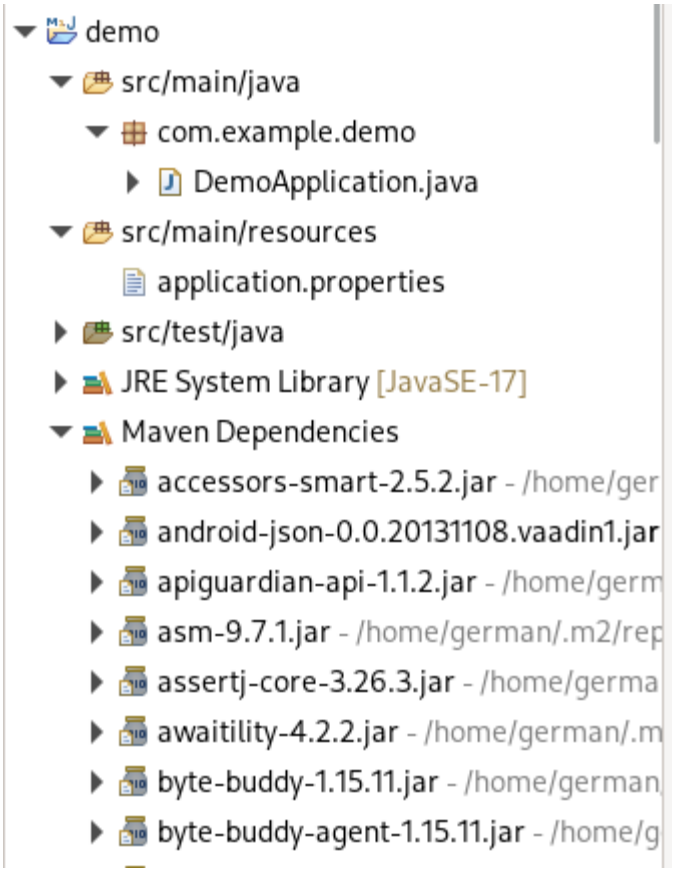
		<dependency>
			<groupId>org.projectlombok</groupId>
			<artifactId>lombok</artifactId>
			<optional>true</optional>
```

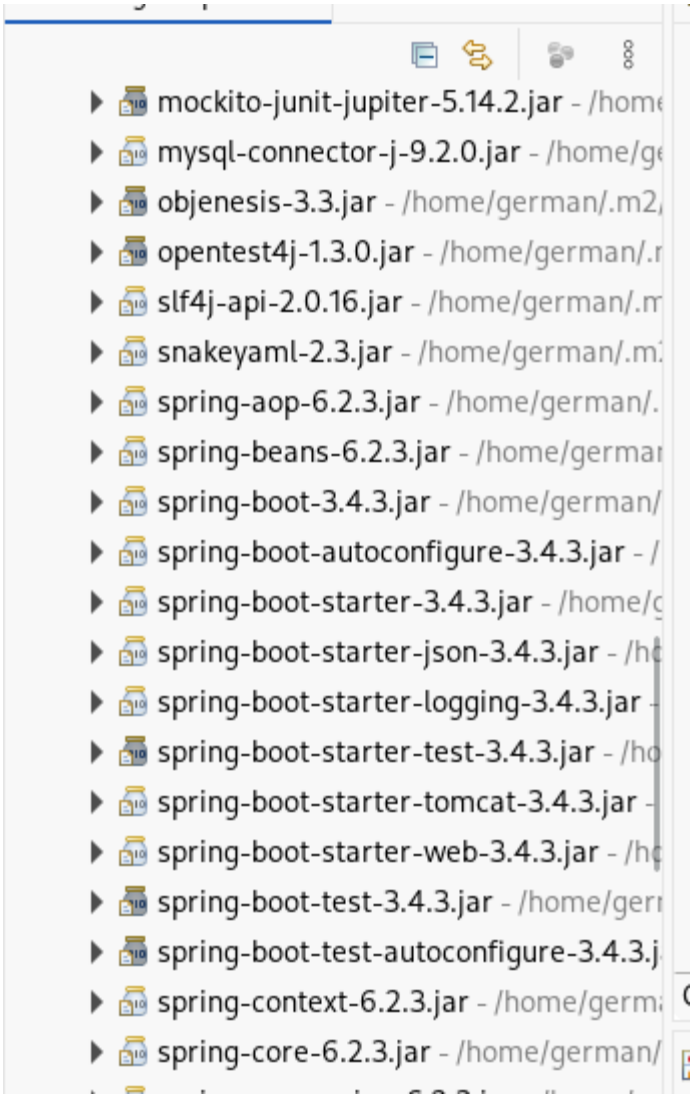
```
            </dependency>
            <dependency>
                  <groupId>org.springframework.boot</groupId>
                  <artifactId>spring-boot-starter-test</artifactId>
                  <scope>test</scope>
            </dependency>
            <dependency>
   <groupId>com.mysql</groupId>
   <artifactId>mysql-connector-j</artifactId>
   <version>9.2.0</version>
</dependency>
      </dependencies>
      <build>
            <plugins>
                  <plugin>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-maven-plugin</artifactId>
                  </plugin>
            </plugins>
      </build>
</project>
```

y hacemos Maven install para instalar los paquetes de nuestro proyecto



Si abrimos Maven Dependencies



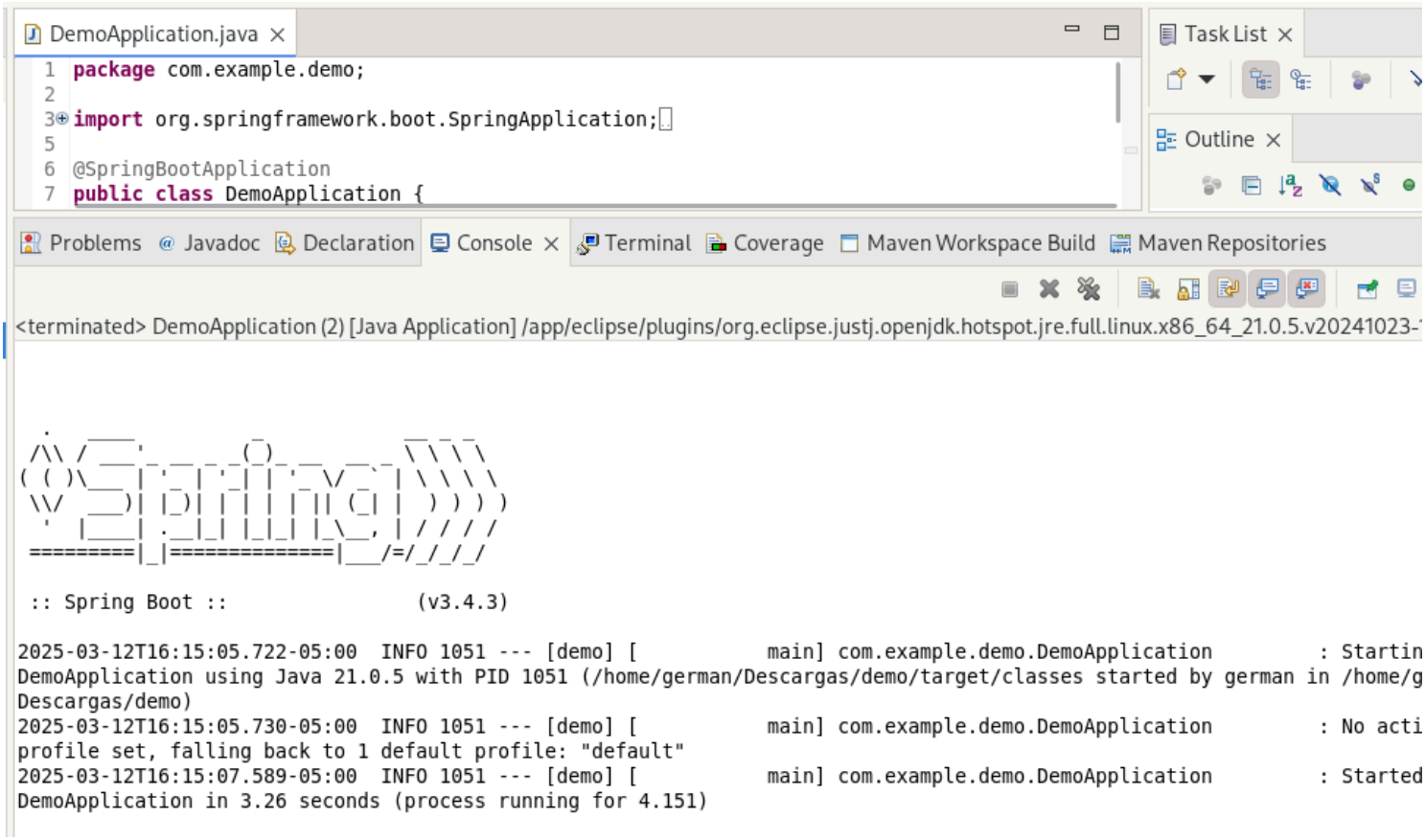


observamos que las librerías de de SpringBoot , Mysql se han descargado e instalado.

Si necesitamos paquetes para nuestras apps con maven, los podemos obtener desde https://mvnrepository.com/

MAVEN NO SOLO SIRVE PARA APLICACIONES SPRINGBOOT , LA PODEMOS USAR PARA FACILITAR LA GESTIÓN DE DEPENDENCIAS DE OTRO TIPO DE PROYECTOS

En el paquete com.example.demo encontramos el punto de entrada de la aplicación, la clase DemoApplication, al ejecutarla obtenemos:



en la consola, SpringBoot nos muestra que se ha ejecutado

Ahora agreguemos las siguientes clases a nuestro proyecto

UsuarioController

```
package com.example.demo;
import java.util.List;
import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestHeader;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@RestController
@RequestMapping("/api/usuarios")
public class UsuarioController {

 private final UsuarioService usuarioService;
  @Autowired
 public UsuarioController(UsuarioService usuarioService) {
     this.usuarioService = usuarioService;
 }
 // Obtener todos los usuarios
 @GetMapping
 public ResponseEntity<List<Usuario>> getAllUsuarios() {
     List<Usuario> usuarios = usuarioService.findAll();
     return new ResponseEntity<>(usuarios, HttpStatus.OK);
 }
 // Buscar por nombre
 @GetMapping("/buscar")
 public ResponseEntity<List<Usuario>> getUsuariosByQuery(@RequestParam String
nombre) {
     List<Usuario> usuarios = usuarioService.findByNombre(nombre);
     return new ResponseEntity<>(usuarios, HttpStatus.OK);
 }
 // Cabeceras
 @GetMapping("/cabecera")
 public ResponseEntity<String> getAgentInfo(@RequestHeader("User-Agent") String
userAgent) {
     String info = "Información del cliente (User-Agent): " + userAgent;
     return new ResponseEntity<>(info, HttpStatus.OK);
 }
 // Obtener un usuario por ID
 @GetMapping("/{id}")
 public ResponseEntity<Usuario> getUsuarioById(@PathVariable String id) {
     Usuario usuario = usuarioService.findById(id);
     if (usuario != null) {
         return new ResponseEntity<>(usuario, HttpStatus.OK);
     } else {
         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
     }
```

```
 }
 // Crear un nuevo usuario
 @PostMapping
 public ResponseEntity<Usuario> createUsuario(@RequestBody Usuario usuario) {
     Usuario newUsuario = usuarioService.save(usuario);
     return new ResponseEntity<>(newUsuario, HttpStatus.CREATED);
 }
 // Actualizar un usuario existente (reemplazo completo)
 @PutMapping("/{id}")
 public ResponseEntity<Usuario> updateUsuario(@PathVariable String id,
@RequestBody Usuario usuario) {
     Usuario existingUsuario = usuarioService.findById(id);
     if (existingUsuario != null) {
         usuario.setId(id);
         Usuario updatedUsuario = usuarioService.update(usuario);
         return new ResponseEntity<>(updatedUsuario, HttpStatus.OK);
     } else {
         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
     }
 }
  // Actualizar solo una parte de los datos
 @PatchMapping("/{id}")
 public ResponseEntity<Usuario> patchUpdateUsuario(@PathVariable String id,
@RequestBody Map<String, Object> updates) {
     Usuario updatedUsuario = usuarioService.patch(id, updates);
     if (updatedUsuario != null) {
         return new ResponseEntity<>(updatedUsuario, HttpStatus.OK);
     } else {
         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
     }
 }
 // Eliminar un usuario
 @DeleteMapping("/{id}")
 public ResponseEntity<Void> deleteUsuario(@PathVariable String id) {
     Usuario existingUsuario = usuarioService.findById(id);
     if (existingUsuario != null) {
         usuarioService.deleteById(id);
         return new ResponseEntity<>(HttpStatus.NO_CONTENT);
     } else {
         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
     }
```

```
  }
}
```

Usuario

```
import java.util.UUID;
public class Usuario {
   private String id;
   private String nombre;
   private String email;
   private int edad;
   public Usuario() {
      this.id = UUID.randomUUID().toString();
   }
   public Usuario(String nombre, String email, int edad) {
      this.id = UUID.randomUUID().toString();
      this.nombre = nombre;
      this.email = email;
      this.edad = edad;
   }
   // Getters y Setters
   public String getId() {
      return id;
   }
   public void setId(String id) {
      this.id = id;
   }
   public String getNombre() {
      return nombre;
   }
   public void setNombre(String nombre) {
      this.nombre = nombre;
   }
   public String getEmail() {
      return email;
   }
   public void setEmail(String email) {
      this.email = email;
```

```
  }
  public int getEdad() {
     return edad;
  }
  public void setEdad(int edad) {
     this.edad = edad;
  }
}
```

UsuarioRepository

```
package com.example.demo;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
import org.springframework.stereotype.Repository;
@Repository
public class UsuarioRepository {

   // Simulamos una base de datos con un Map
   private final Map<String, Usuario> baseDeDatos = new HashMap<>();

   // Guardar un usuario
   public Usuario save(Usuario usuario) {
       baseDeDatos.put(usuario.getId(), usuario);
       return usuario;
   }

   // Encontrar usuario por ID
   public Usuario findById(String id) {
       return baseDeDatos.get(id);
   }

   // Listar todos los usuarios
   public List<Usuario> findAll() {
       return new ArrayList<>(baseDeDatos.values());
   }
```

```
    // Filtrar por nombre
    public List<Usuario> findByNombreContaining(String nombre) {
        return baseDeDatos.values().stream()
                .filter(usuario ->
usuario.getNombre().toLowerCase().contains(nombre.toLowerCase()))
                .collect(Collectors.toList());
    }

    // Eliminar un usuario
    public void deleteById(String id) {
        baseDeDatos.remove(id);
    }

    // Actualizar un usuario
    public Usuario update(Usuario usuario) {
        if (baseDeDatos.containsKey(usuario.getId())) {
            baseDeDatos.put(usuario.getId(), usuario);
            return usuario;
        }
        return null;
    }
}
```

UsuarioService

```
package com.example.demo;
import java.util.List;
import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    @Autowired
    public UsuarioService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
```

```
        // Inicializamos algunos datos de ejemplo
        initSampleData();
    }

    private void initSampleData() {
        save(new Usuario("Juan Pérez", "juan@example.com", 30));
        save(new Usuario("María López", "maria@example.com", 25));
        save(new Usuario("Carlos Ruiz", "carlos@example.com", 40));
    }

    // Crear un nuevo usuario
    public Usuario save(Usuario usuario) {
        return usuarioRepository.save(usuario);
    }

    // Obtener un usuario por ID
    public Usuario findById(String id) {
        return usuarioRepository.findById(id);
    }

    // Listar todos los usuarios
    public List<Usuario> findAll() {
        return usuarioRepository.findAll();
    }

    // Buscar por nombre
    public List<Usuario> findByNombre(String nombre) {
        return usuarioRepository.findByNombreContaining(nombre);
    }

    // Actualizar un usuario
    public Usuario update(Usuario usuario) {
        return usuarioRepository.update(usuario);
    }

    // Actualización parcial
    public Usuario patch(String id, Map<String, Object> updates) {
        Usuario usuario = usuarioRepository.findById(id);
        if (usuario != null) {
            updates.forEach((key, value) -> {
                switch (key) {
```

```
                    case "nombre":
                        usuario.setNombre((String) value);
                        break;
                    case "email":
                        usuario.setEmail((String) value);
                        break;
                    case "edad":
                        usuario.setEdad((Integer) value);
                        break;
                }
            });
            return usuarioRepository.update(usuario);
        }
        return null;
    }

    // Eliminar un usuario
    public void deleteById(String id) {
        usuarioRepository.deleteById(id);
    }
}
```

Ahora nuestro proyecto se ve así

Ejecutamos nuestro proyecto y ya tenemos nuestra app funcionando con las nuevas clases agregadas

UsuarioControll | DemoApplication | UsuarioReposito | UsuarioService.

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

Problems | Javadoc | Declaration | Console | Terminal | Coverage | Maven Workspace Build

DemoApplication (2) [Java Application] /app/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x86_64_21.0

```
 \ \ /\___ |  _ |  _| |  _ \/  _  | \ \ \ \
  \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
   '  |____| .__|_| |_|_| |_\__, | / / / /
  =========|_|==============|___/=/_/_/_/

  :: Spring Boot ::                (v3.4.3)

2025-03-12T17:33:57.657-05:00  INFO 3115 --- [demo] [           main] com.example.demo.DemoAppli
DemoApplication using Java 21.0.5 with PID 3115 (/home/german/Descargas/demo/target/classes star
Descargas/demo)
2025-03-12T17:33:57.660-05:00  INFO 3115 --- [demo] [           main] com.example.demo.DemoAppli
profile set, falling back to 1 default profile: "default"
```